

Call Graph and Coupling

NexusNotes

Centralized Academic Resource Platform

Project Team: Code Club

Ramasani Adi Pranav – 24CSB0A57

Syed Mohammad Sami – 24CSB0A77

Janapala Harsha Vardhan Reddy – 24CSB0A28

Version 1.0

March 7, 2026

Contents

1	Introduction	2
2	Source Code	3
2.1	Student Controller	3
2.2	Admin Controller	4
2.3	Authentication Controller	4
3	Call Graph Diagrams	5
3.1	Student Module Call Graph	5
3.2	Student Module Call Graph	6
3.3	Admin Module Call Graph	7
4	Module Structure Diagram	9
5	Coupling and Cohesion Analysis	11
5.1	Cohesion	11
5.2	Coupling	12
6	Conclusion	13

Chapter 1

Introduction

NexusNotes is a centralized academic resource management platform designed to organize and distribute academic materials efficiently. The system allows students to upload, search, and download academic resources while administrators monitor and manage the platform.

The system architecture consists of several modules including authentication, resource management, search and retrieval, and administration. These modules interact to ensure secure access, structured resource storage, and efficient information retrieval.

This document presents the source code structure, call graphs, module structure, and coupling relationships of the NexusNotes platform.

Chapter 2

Source Code

2.1 Student Controller

```
exports.uploadAcademicResource = async (req, res) => {
  const { title, subject, category, tags } = req.body;
  const file = req.file;

  if (!validateFile(file)) {
    return res.status(400).json({ error: 'Invalid file' });
  }

  const resource = await saveResource(
    req.user.id,
    title,
    subject,
    category,
    tags,
    file.path
  );

  res.status(201).json({ success: true, resource });
};

function validateFile(file){
  return file.size <= 50*1024*1024;
}

async function saveResource(){
  // store resource
}
```

2.2 Admin Controller

```
exports.manageUserRoles = async (req,res)=>{

  const {targetUserId,newRole} = req.body;

  if(!verifyAdminPrivileges(req.user)){
    return res.status(403).json({error:"Unauthorized"});
  }

  const updatedUser =
    await updateUserRole(targetUserId,newRole);

  res.status(200).json({success:true,updatedUser});
}

function verifyAdminPrivileges(user){
  return user.role === 'Administrator';
}
```

2.3 Authentication Controller

```
exports.login = async(req,res)=>{

  const {email,password} = req.body;

  const user = await checkCredentials(email,password);

  if(user){
    const token = generateToken(user._id,user.role);
    res.status(200).json({token});
  }
}

function generateToken(id,role){
  return jwt.sign({id,role},"SECRET",{expiresIn:"1d"});
}
```

Chapter 3

Call Graph Diagrams

Call graphs illustrate how functions within the system interact with each other during execution.

3.1 Student Module Call Graph

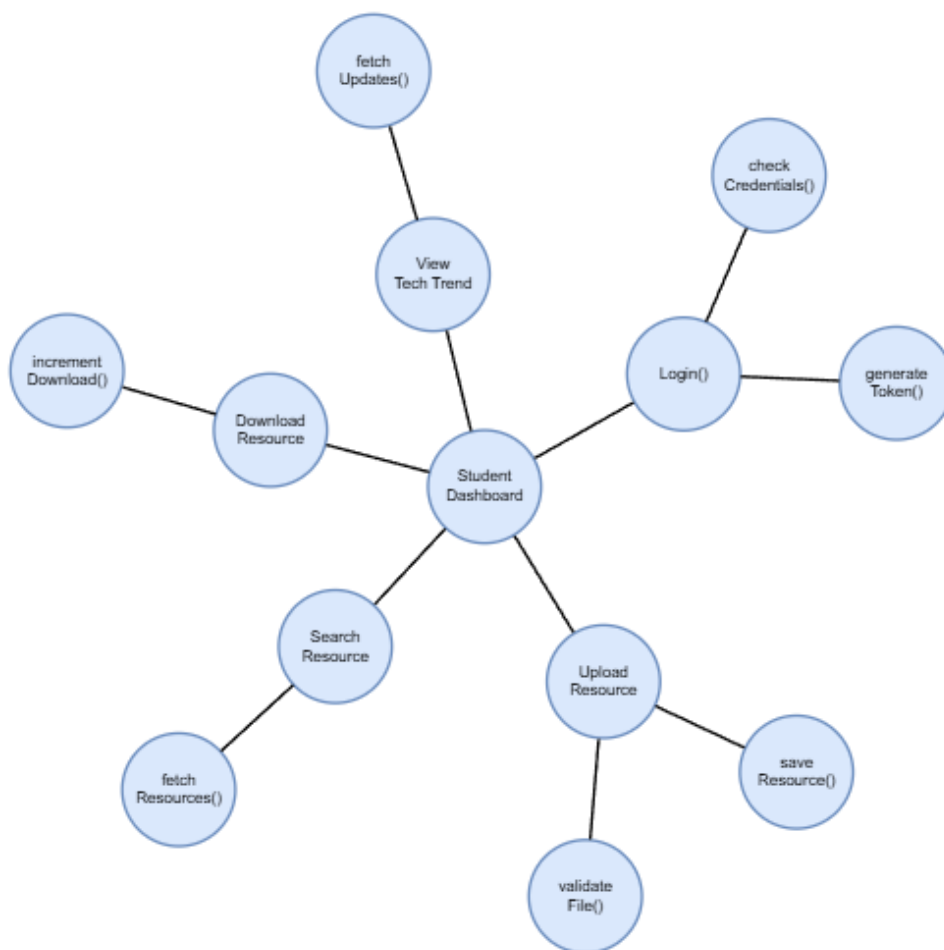


Figure 3.1: Student Module Call Graph

3.2 Student Module Call Graph

1. Student Dashboard as the Primary Controller

- The Student Dashboard acts as the central interface through which students interact with the NexusNotes platform.
- It connects all major functionalities such as uploading resources, searching materials, downloading files, and viewing technology updates.
- This centralized structure simplifies user interaction and ensures organized navigation within the system.

2. Authentication and Login Process

- The Login() function initiates the authentication process for the student.
- The system calls checkCredentials() to verify whether the entered email and password match the stored user data.
- Once verification is successful, generateToken() creates an authentication token that allows secure access to system features.
- This process ensures that only authorized students can upload or access academic resources.

3. Resource Upload Process

- Upload Resource is responsible for allowing students to contribute academic materials such as lecture notes, project reports, or presentations.
- The validateFile() function checks whether the uploaded file meets system requirements such as supported formats and size limits.
- Once validated, saveResource() stores the resource in the database along with its metadata including title, subject, category, and tags.
- This ensures that uploaded resources are properly stored and categorized within the repository.

4. Resource Search and Retrieval

- Search Resource allows students to locate academic materials using keywords or filters.
- The system calls fetchResources() to retrieve relevant materials from the database.
- The retrieved resources are displayed to the student based on relevance and availability.
- This functionality ensures quick access to study materials stored in the system.

5. Resource Download Process

- Download Resource enables students to obtain academic files for offline study.
- The incrementDownload() function updates the download count of the resource.
- Tracking download statistics helps the system identify popular and frequently accessed resources.

6. Technology Trend Feed

- View Tech Trend allows students to stay updated with current developments in technology related to their academic domain.
- The fetchUpdates() function retrieves curated technology news and updates from external APIs or services.
- This feature bridges the gap between academic learning and industry trends.

3.3 Admin Module Call Graph

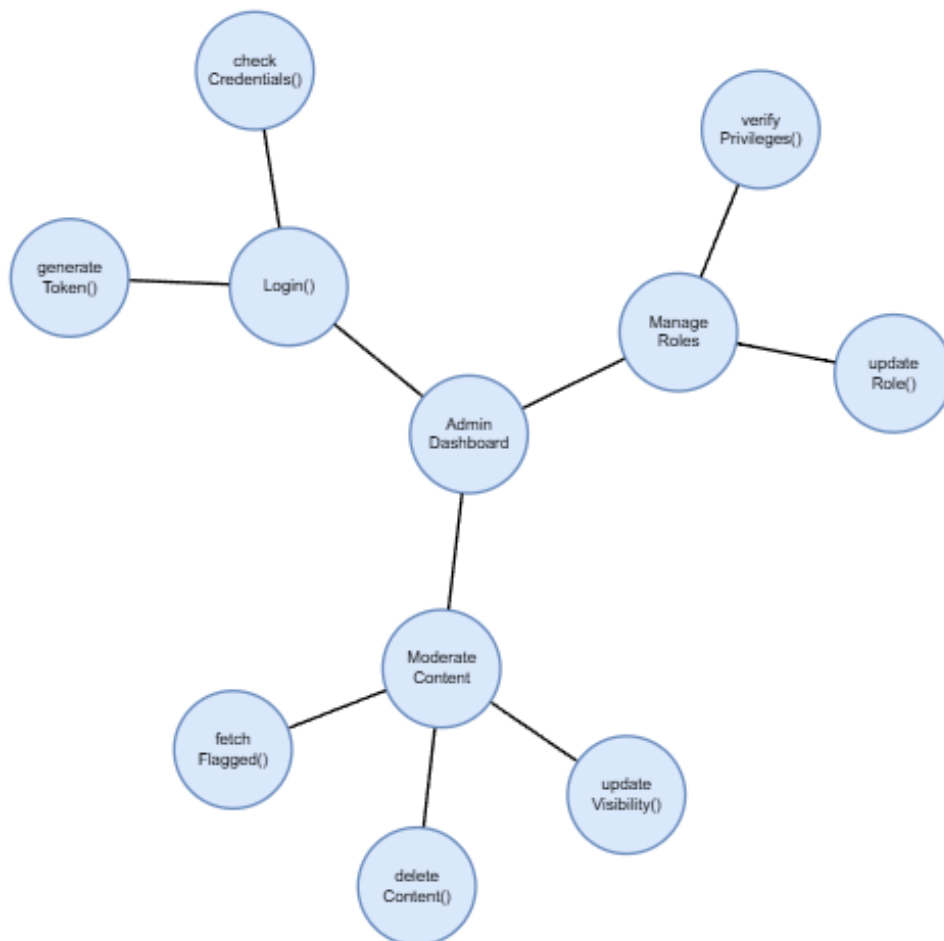


Figure 3.2: Admin Module Call Graph

1. Admin Dashboard as the Central Control Unit

- The Admin Dashboard serves as the main control interface for administrative activities.
- It connects administrative tasks such as role management, content moderation, and system monitoring.
- This centralized structure enables administrators to efficiently manage the platform.

2. Authentication and Security

- The `Login()` function begins the administrator authentication process.
- The system verifies credentials using `checkCredentials()`.
- If authentication is successful, `generateToken()` generates a secure token that grants access to administrative functionalities.
- This ensures that only authorized administrators can modify system data.

3. User Role Management

- Manage Roles allows administrators to assign or modify roles of system users.
- `verifyPrivileges()` ensures that the current user has administrator permissions.
- `updateRole()` updates the role information of the selected user in the database.
- This functionality helps maintain proper access control across the system.

4. Content Moderation

- Moderate Content allows administrators to review resources uploaded by students.
- `fetchFlagged()` retrieves content that has been reported or flagged for review.
- `deleteContent()` removes inappropriate or invalid resources from the system.
- `updateVisibility()` changes the visibility status of resources, allowing administrators to approve or restrict access.
- This ensures that only verified and relevant academic materials remain available on the platform.

Chapter 4

Module Structure Diagram

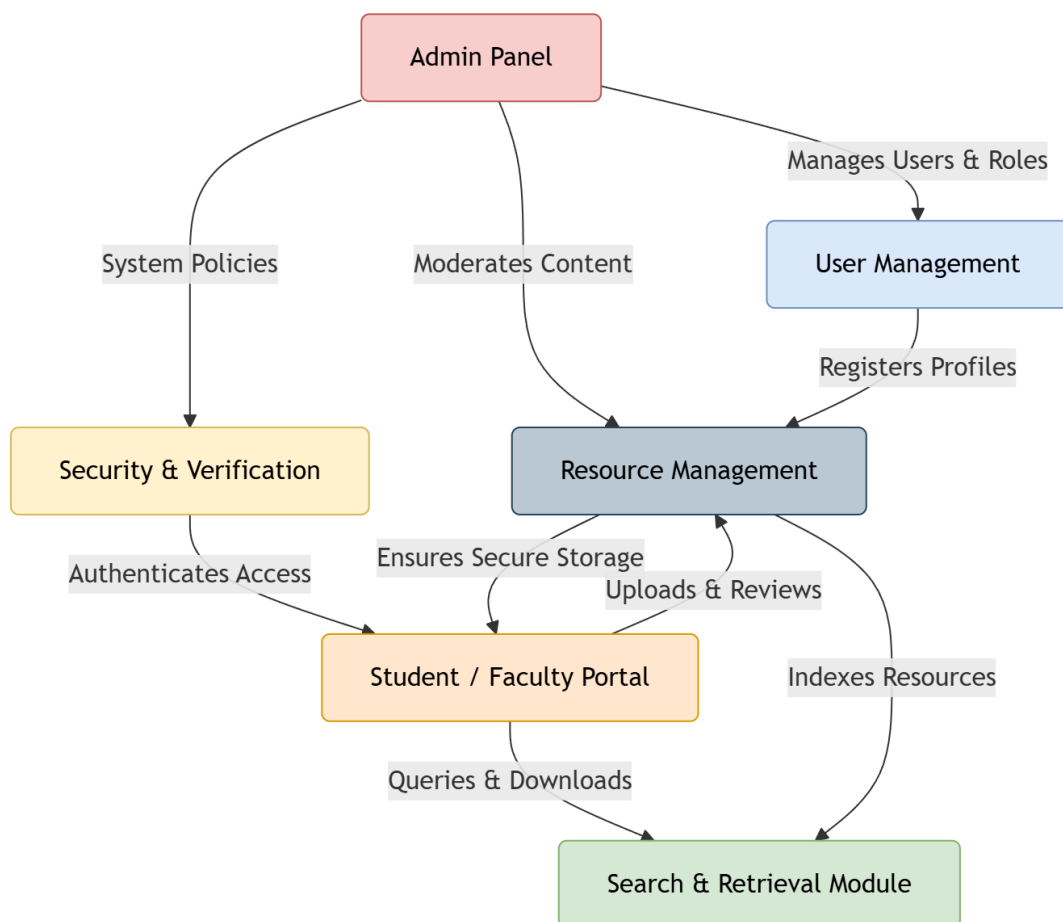


Figure 4.1: Module Structure of NexusNotes

Explanation

1. Security Module

- The Security Module manages authentication and access control for users.
- Authentication verifies the identity of users before granting system access.
- Security verification ensures that only authorized individuals can interact with the platform.
- This module protects the system from unauthorized usage.

2. Admin Module

- The Admin Module manages platform operations and governance.
- Admin Panel acts as the central interface for administrators.
- User Management controls user roles and permissions.
- Content Moderation ensures that uploaded resources meet academic standards.

3. Resource Module

- The Resource Module handles all operations related to academic materials.
- Upload Resource allows users to contribute academic files.
- Download Resource enables students to retrieve study materials.
- Resource Operations organize and manage stored academic content.

4. Search and Retrieval Module

- The Search Engine processes user queries.
- Query Processing interprets search keywords and filters.
- Content Indexing organizes stored resources to enable fast retrieval.
- This module ensures efficient searching of academic materials.

Chapter 5

Coupling and Cohesion Analysis

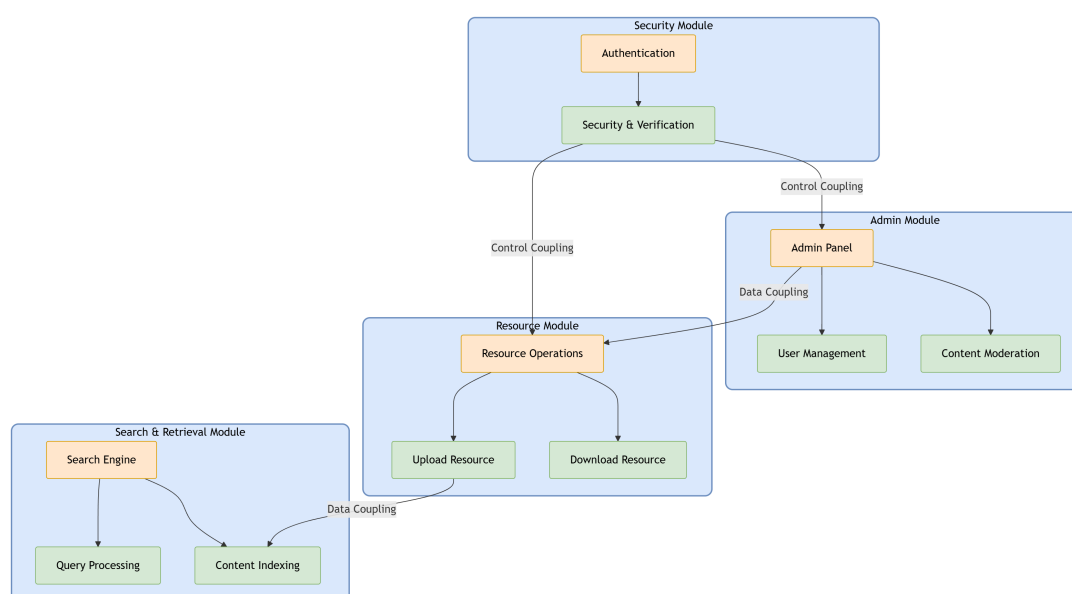


Figure 5.1: System Coupling Diagram

5.1 Cohesion

Cohesion refers to the degree to which the elements within a module belong together.

- Each module in NexusNotes performs a specific and well-defined function.
- Security Module focuses only on authentication and verification.
- Admin Module manages system governance and user roles.
- Resource Module handles uploading, downloading, and storage of academic materials.
- Search Module manages searching and indexing operations.

Benefits of High Cohesion

- Improves maintainability of the system.
- Allows independent modification of modules.
- Enhances code readability and debugging efficiency.

5.2 Coupling

Coupling refers to the level of dependency between different modules.

Control Coupling

- Security Module → Admin Module Authentication controls access to administrative functionalities.
- Security Module → Resource Module User verification determines whether a user can upload or download resources.

Data Coupling

- Resource Module → Search Module Uploaded resources are indexed for search operations.
- Admin Module → Resource Module Administrative moderation actions update resource visibility and availability.

Impact of Coupling

- Controlled module interaction maintains system integrity.
- Data coupling ensures efficient information exchange between modules.
- Balanced coupling improves modularity while preserving functionality.

Chapter 6

Conclusion

The NexusNotes system demonstrates a modular architecture with clear separation of responsibilities across different components. The call graph diagrams illustrate how system functions interact, while the module structure diagram shows the overall organization of the platform.

Through effective cohesion and controlled coupling between modules, the system ensures maintainability, scalability, and secure management of academic resources.